GENERATORS IN PYTHON

Generator functions, yield, next(), and the generator process

Learning objective Understand how Python generators produce values one at a time, how yield pauses and resumes function execution, and how generators can be used to process data efficiently.

1. What Is a Generator?

A generator is a special type of Python function that produces values one at a time instead of creating and storing all values together.

A normal function can create a list and return all the values at once. A generator produces each value when it is requested.

```
def get_numbers():
    yield 10
    yield 20
    yield 30

numbers = get_numbers()
print(numbers)
```

Typical output:

```
<generator object get_numbers at 0x...>
```

The call get_numbers() creates a generator object. The function body has not started executing yet.

Key pointA generator produces values one at a time. It does not need to build and store the complete collection first.

2. Understanding the yield and next () keywords

The yield keyword produces a value and temporarily pauses the function. Python remembers where the function stopped and continues from that point when another value is requested. The next() function requests the next value from a generator. Each call resumes the generator from where it previously paused

```
def get_numbers():
    yield 10
    yield 20
    yield 30

numbers = get_numbers()

print(next(numbers))
print(next(numbers))
print(next(numbers))
```

Output:

```
10
20
30
```

The first next() call returns 10, the second returns 20, and the third returns 30. The generator remembers its position after each call.

A for loop can retrieve all generated values automatically. It repeatedly requests the next value and stops when the generator is exhausted.

When Python reaches yield, it returns the value and pauses. The next request resumes execution from the statement immediately after the previous yield.

3. Retrieving Values using for Loop

```
.def get_numbers():
    yield 10
    yield 20
    yield 30

numbers = get_numbers()

for num in numbers
    print(num)
```

Output:

```
10
20
30
```

Practical guidance Use a for loop to process all generated values. Use next() when you want to retrieve values manually, one at a time.

4. Difference Between return and yield

Both return and yield provide a value, but they behave differently.

Both return and yield can provide a value, but they have different effects on function execution.

return	yield
Returns a value and terminates the function.	Produces a value and pauses the generator function.
A later statement in the function is not executed.	Execution can continue from the point after yield.
Used in ordinary functions.	Used to define generator functions.
A normal function call returns its result.	A generator function call creates a generator object.

5. Practical Example: Generating Even Numbers

A generator can use loops and conditions. It produces a value only when the condition is satisfied.

```
def even_numbers():
    for number in range(1, 11):
        if number % 2 == 0:
            yield number

for value in even_numbers():
    print(value)
```

Output:

```
2
4
6
8
10
```

6. What Happens When a Generator Is Exhausted?

When a generator has produced all its values, it is called an exhausted generator. Calling next() again raises StopIteration because there are no more values to provide.

```
def get_values():
    yield "A"
    yield "B"

values = get_values()

print(next(values))
print(next(values))
print(next(values))  # Raises StopIteration
```

Output before the exception:

```
A
B
```

A for loop handles the end of a generator automatically. We normally do not need to catch StopIteration when using a for loop.

RememberAn exhausted generator does not automatically restart. Call the generator function again to create a new generator object.

7. Why Are Generators Useful?

A list stores all its elements in memory. A generator produces values when they are requested. This can reduce memory use when working with large sequences, especially when values can be processed one at a time.

Example: generating numbers up to one million without creating a list.

```
def generate_numbers():
    for number in range(1, 1000001):
        yield number

total = 0
for value in generate_numbers():
    total += value

print("Total:", total)
```

Output:

```
Total: 500000500000
```

8. Practice Exercises

Exercise 1 – Numbers from 1 to 10: Create a generator function that yields numbers from 1 to 10. Display the values using a for loop.

Exercise 2 – Odd numbers: Create a generator that produces all odd numbers between 1 and 20.

Exercise 3 – Multiplication table: Create a generator that yields the first 10 multiples of a number supplied as an argument. Test it with 5.

Exercise 4 – Names beginning with A: Given a list of names, create a generator that produces only names beginning with the letter A.

Exercise 5 – Using next(): Create a generator that yields 100, 200 and 300. Retrieve each value using next(). What happens when you request one more value?

9. Quick Recap

GeneratorA special function that produces values one at a time.

yield	Produces a value and pauses the generator function.
Generator object	Created when a generator function is called; values are produced when requested.
next()	Requests the next value from a generator.
StopIteration	Indicates that a generator has no more values.
for loop	Retrieves generated values automatically until the generator is exhausted.